

Nicolás Aguirre Velásquez
Jhon Freddy Patino Meza
Gabriel Esteban Huertas Ruiz
Diego Alexander Ibanez Torres

Introducción

Este tutorial muestra cómo construir una aplicación backend básica utilizando Spring Boot y PostgreSQL, separando responsabilidades entre base de datos y aplicación. La base de datos se ejecuta dentro de un contenedor Docker para facilitar la instalación y asegurar que el entorno sea reproducible, mientras que el backend se ejecuta localmente para enfocarse primero en la lógica de desarrollo sin añadir complejidad extra.

El objetivo es conectar una API REST en Java con una base de datos PostgreSQL, consultar información almacenada en una tabla y exponerla mediante un endpoint HTTP consumible desde navegador o Postman.

Objetivos

- Instalar y verificar las herramientas necesarias para el desarrollo.
- Crear una base de datos PostgreSQL usando Docker Compose.
- Inicializar una tabla y poblar con datos mediante scripts SQL.
- Crear un backend con Spring Boot usando arquitectura por capas.
- Conectar Spring Boot con PostgreSQL mediante JPA/Hibernate.
- Exponer un endpoint REST y probarlo desde navegador y Postman.

Prerrequisitos

Este tutorial fue realizado y probado en el sistema operativo Ubuntu, por lo que los comandos mostrados corresponden a ese entorno. En otras distribuciones Linux, Windows o macOS algunos pasos pueden variar, especialmente los relacionados con instalación de paquetes y rutas del sistema.

Para seguir correctamente el tutorial se recomienda contar con:

- Un equipo con Ubuntu instalado.
- Conexión a internet para descargar dependencias e imágenes Docker.
- Permisos de administrador (sudo) para instalar paquetes.
- Conocimientos básicos de terminal Linux.
- Espacio disponible para instalar Docker, Java y dependencias del proyecto.

Para comenzar, actualizamos el sistema.

```
sudo apt update && sudo apt upgrade -y
```

- Docker

Docker permite ejecutar servicios en contenedores aislados. En este tutorial se usa para PostgreSQL porque evita instalar manualmente el motor en el sistema operativo y simplifica la configuración entre distintos equipos.

```
sudo apt install docker.io docker-compose-v2 -y
```

```
docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2

docker compose version
Docker Compose version 2.40.3+ds1-0ubuntu1~24.04.1

sudo docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working
correctly.
To generate this message,...
```

- Java JDK

Spring Boot está desarrollado en Java, por lo que se necesita el JDK (Java Development Kit), que incluye tanto el runtime como el compilador (javac).

```
sudo apt install openjdk-25-jdk -y
```

```
java --version
openjdk version "25.0.2" 2026-01-20
OpenJDK Runtime Environment...

javac --version
javac 25.0.2
```

- Maven

Aunque no es obligatorio instalar Maven globalmente, el proyecto generado por Spring Initializr incluye mvnw (Maven Wrapper), que descarga y ejecuta Maven automáticamente.

- Postman

Permite probar endpoints HTTP sin depender del navegador, facilitando la inspección de respuestas JSON y códigos de estado.

- Editor de código

Puede utilizarse cualquier editor o entorno de desarrollo compatible con proyectos Java, como Visual Studio Code, Eclipse IDE o editores similares. Sin embargo, se recomienda trabajar con IntelliJ IDEA, ya que integra soporte nativo para proyectos de Spring Boot, facilita la detección de dependencias Maven y simplifica la ejecución del backend.

- Navegador web

Puede utilizarse cualquier navegador moderno, como Google Chrome, Mozilla Firefox, Microsoft Edge o similares. En este tutorial se emplea únicamente para acceder al endpoint HTTP generado por el backend y visualizar la respuesta JSON.

Base de datos (con docker)

Primero se comenzará creando e ingresando a la carpeta del proyecto.

```
mkdir holamundo
cd holamundo
```

Ya estando dentro, vamos a crear el archivo docker-compose.yml. Docker Compose permite definir servicios completos en un solo archivo declarativo.

En este caso se utiliza para describir qué imagen (versión) usar, qué puerto exponer, qué credenciales iniciales crear y dónde persistir los datos, logrando que cualquier persona pueda levantar la base con un solo comando.

```
nano docker-compose.yml
```

Luego, dentro de él vamos a colocar lo siguiente:

```
services:
  db:
    container_name: pg-demo-container
    image: postgres:16
    ports:
      - "${DB_PORT}:5432"
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Salimos con CTRL+O, ENTER, CTRL+X.

Cada sección tiene la siguiente función:

services: define los contenedores del proyecto.
db: nombre lógico del servicio de base de datos.
image: imagen oficial de PostgreSQL que descargará Docker.
ports: conecta el puerto del contenedor con el del host.
environment: variables que PostgreSQL usa al iniciar.
volumes: permite conservar datos aunque el contenedor se elimine.

Ahora vamos a crear el archivo `.env`. Este archivo permite separar variables de configuración del archivo `docker-compose.yml`. Como ventaja tiene que evita repetir valores sensibles como usuario y contraseña, facilita cambiar puertos o nombres de base de datos sin editar el `compose`, y mejora la portabilidad entre diferentes máquinas. En proyectos reales, este patrón permite manejar configuraciones distintas para desarrollo, pruebas y producción.

```
nano .env
```

Dentro de él vamos a colocar lo siguiente:

```
# Base de datos PostgreSQL
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=postgres123
DB_NAME=holaamundo
```

Salimos con CTRL+O, ENTER, CTRL+X

Una vez realizados los pasos anteriores, ya se puede levantar el contenedor con nuestra base de datos, con la flag `-d` le decimos que lo haga en segundo plano:

```
docker compose up -d
```

Se puede verificar que todo está bien con:

```
docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS        PORTS          NAMES
1669a90fa7a3   postgres:16    "docker-entrypoint.s..." 3 seconds
ago           Up 2 seconds   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
pg-demo-container
```

Ahora vamos a ingresar a la base de datos que se crea por defecto usando:

```
docker exec -it pg-demo-container psql -U postgres -d postgres
```

Estamos accediendo a postgresql dentro de nuestro contenedor pg-demo-container. Además estamos accediendo a la base de datos postgres usando el usuario postgres.

Desde aquí vamos a crear nuestra base de datos y a conectarnos a ella.:

```
postgres# \c holamundo;
holamundo# \q
```

De ahora en adelante, nos conectaremos a ella con:

```
docker exec -it pg-demo-container psql -U postgres -d holamundo
```

Ahora crearemos una carpeta llamada scripts, en dónde estarán los scripts de sql necesarios. En nuestro caso crearemos uno llamado holamundo.sql:

```
mkdir scripts
nano scripts/holamundo.sql
```

Este script creará una única tabla en la base de datos e insertará algunos valores, por lo que le colocamos lo siguiente:

```
CREATE TABLE frase (
    id SERIAL PRIMARY KEY,
    contenido TEXT
);

INSERT INTO frase(contenido)
VALUES
('Hola mundo'),
('Viva docker'),
('Oscar el mejor profe');
```

Salimos con CTRL+O, ENTER, CTRL+X

Y por último, corremos dicho script así:

```
docker exec -i pg-demo-container psql -U postgres -d holamundo <
scripts/holamundo.sql
```

Aquí estamos ejecutando el script holamundo.sql, usando postgresql dentro de nuestro contenedor pg-demo-container. Además, lo estamos haciendo en la base de datos holamundo usando el usuario postgres.

Podemos revisar que todo va bien conectándonos a la base de datos y realizando una consulta:

```
docker exec -it pg-demo-container psql -U postgres -d holamundo
```

```
holamundo# SELECT * FROM frase;
```

```
id |      contenido
----+-----
 1 | Hola mundo
 2 | Viva docker
 3 | Oscar el mejor profe
(3 rows)
```

NO lo haremos por ahora, pero el contenedor se puede apagar con:

```
docker compose down
```

Backend (con Spring Initializr, sin docker)

Para el caso de Spring Boot, existe una herramienta llamada [Spring Initializr](#) que permite inicializar proyectos. En la imagen se muestra la configuración inicial del proyecto backend utilizando Spring Initializr, una herramienta que permite generar automáticamente la estructura base de un proyecto en Spring Boot.

The screenshot shows the Spring Initializr web application interface. The 'Project' section has 'Maven' selected. The 'Language' section has 'Java' selected. The 'Spring Boot' section has '4.0.6' selected. The 'Project Metadata' section shows 'Group' as 'com.example', 'Artifact' as 'backend', and 'Package name' as 'com.example.backend'. The 'Packaging' section has 'Jar' selected, and the 'Configuration' section has 'Properties' selected. The 'Java' version is set to '25'. The 'Dependencies' section on the right lists 'Spring Web' (WEB), 'Spring Data JPA' (SQL), and 'PostgreSQL Driver' (SQL). At the bottom, there are buttons for 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and a menu button '...'.

Se selecciona Apache Maven como sistema de construcción porque permite gestionar dependencias y ejecutar el proyecto fácilmente, además de generar automáticamente el archivo pom.xml con la configuración necesaria. Como lenguaje se utiliza Java, ya que es la base de Spring Boot, y en este caso se eligió Java 25 por ser compatible con la versión seleccionada y la versión LTS más reciente. La versión de Spring Boot se escogió por ser estable, reciente y adecuada para desarrollar aplicaciones backend de forma sencilla.

Finalmente, se agregan las dependencias necesarias para este tutorial:

- **Spring Web:** permite crear endpoints REST y exponer servicios HTTP.
- **Spring Data JPA:** facilita la interacción con la base de datos mediante ORM, usando Hibernate.
- **PostgreSQL Driver:** proporciona el conector JDBC para que la aplicación pueda comunicarse con la base de datos PostgreSQL.

Estas dependencias permiten construir un backend capaz de consultar la base de datos y exponer los resultados a través de una API REST.

Al darle en generar, se descarga un archivo comprimido backend.zip. Este archivo debemos descomprimirlo en la carpeta del proyecto holamundo/ y verificar que dentro de backend exista por lo menos esta estructura:

```

.
├── HELP.md
├── mvnw
├── mvnw.cmd
├── pom.xml
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── example
│   │   │   │   │   ├── backend
│   │   │   │   │   │   └── BackendApplication.java
│   │   └── resources
│   │       ├── application.properties
│   │       ├── static
│   │       └── templates
└──

```

Ahora, buscaremos el archivo application.properties, este archivo contiene la configuración principal del backend Spring Boot. Se usa para definir cómo la aplicación se conecta a la base de datos y cómo se comporta Hibernate. Dentro colocaremos lo siguiente:

```

spring.application.name=backend

spring.datasource.url=jdbc:postgresql://localhost:5432/holamundo
spring.datasource.username=postgres
spring.datasource.password=postgres123

spring.jpa.hibernate.ddl-auto=none spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect

```

spring.datasource.url: Indica la dirección JDBC que Spring usará para conectarse a PostgreSQL.

spring.datasource.username: Usuario con el que la aplicación se autenticará en la base de datos.

spring.datasource.password: Contraseña del usuario.

spring.jpa.hibernate.ddl-auto=none: Indica que Hibernate no debe crear ni modificar tablas automáticamente, ya que la estructura fue creada manualmente mediante scripts SQL.

spring.jpa.show-sql=true: Permite visualizar en consola las consultas SQL generadas por Hibernate.

Además, usaremos la siguiente arquitectura por capas:

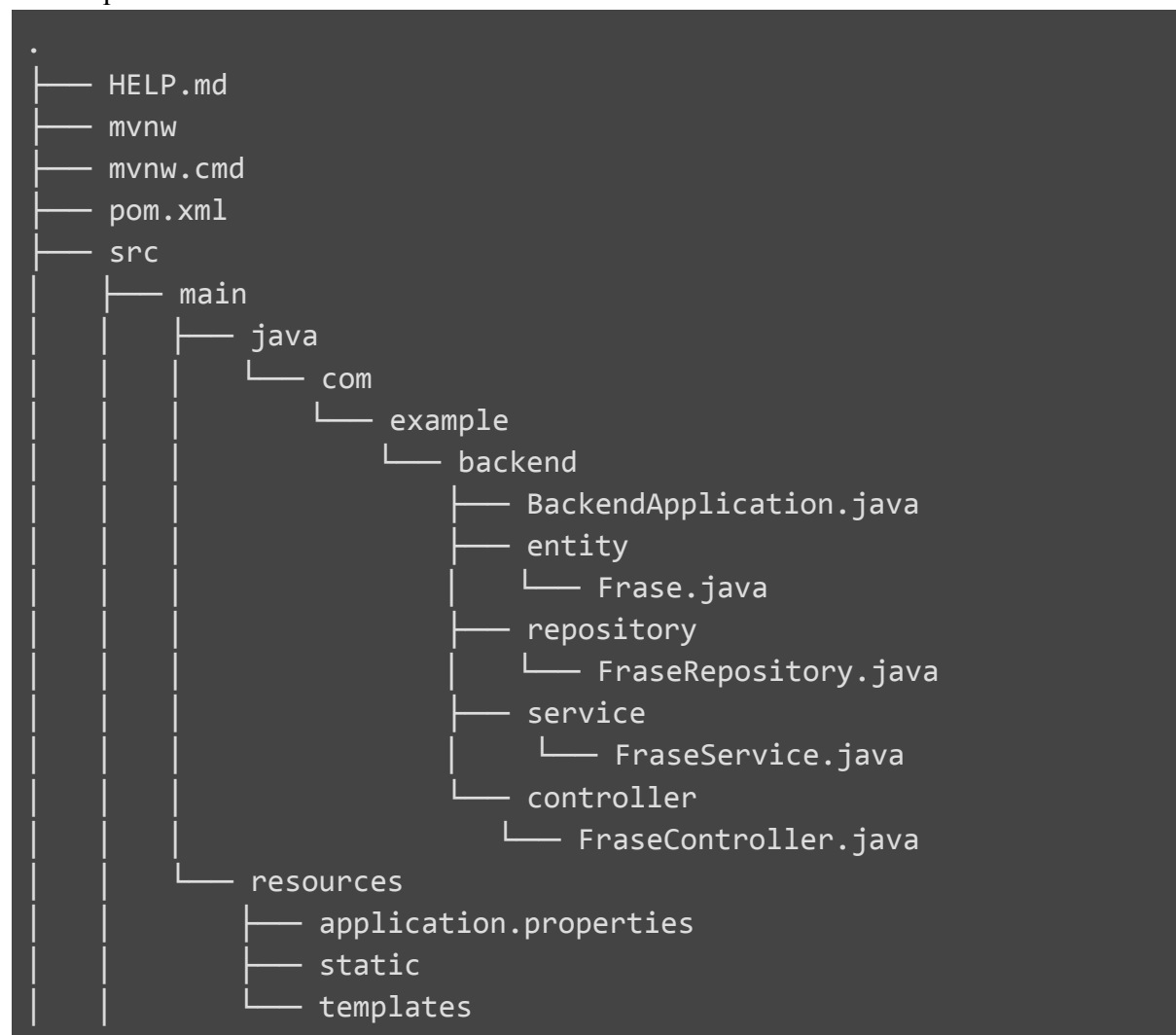
Controller: Gestiona las solicitudes HTTP que llegan al backend.

Service: Contiene la lógica de negocio y actúa como intermediario entre controller y repository.

Repository: Se comunica directamente con la base de datos usando JPA.

Entity: Representa tablas de la base de datos como clases Java.

Por lo que nuestra estructura ahora se verá así:



Y los archivos de cada paquete quedan así:

entity/Frase.java

```
package com.example.backend.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "frase")
public class Frase {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String contenido;

    public Long getId() {
        return id;
    }

    public String getMensaje() {
        return contenido;
    }

    public void setMensaje(String contenido) {
        this.contenido = contenido;
    }
}
```

repository/FraseRepository.java

```
package com.example.backend.repository;

import com.example.backend.entity.Frase;
import org.springframework.data.jpa.repository.JpaRepository;

public interface FraseRepository extends JpaRepository<Frase, Long> {
}
```

service/FraseService.java

```
package com.example.backend.service;

import com.example.backend.entity.Frase;
import com.example.backend.repository.FraseRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class FraseService {

    private final FraseRepository fraseRepository;

    public FraseService(FraseRepository fraseRepository) {
        this.fraseRepository = fraseRepository;
    }

    public List<Frase> obtenerTodas() {
        return fraseRepository.findAll();
    }
}
```

controller/FraseController.java

```
package com.example.backend.controller;

import com.example.backend.entity.Frase;
import com.example.backend.service.FraseService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class FraseController {

    private final FraseService fraseService;

    public FraseController(FraseService fraseService) {
        this.fraseService = fraseService;
    }
}
```

```

@GetMapping("/frases")
public List<Frase> listarFrases() {
    return fraseService.obtenerTodas();
}
}

```

BackendApplication.java

```

package com.example.backend;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BackendApplication {

    public static void main(String[] args) {
        SpringApplication.run(BackendApplication.class, args);
    }

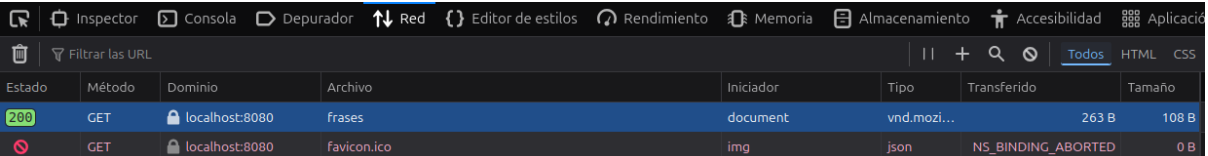
}

```

Finalmente, podemos correrlo dándole run al archivo BackendApplication.java desde IntelliJ, o con el siguiente comando:

```
./mvnw spring-boot:run
```

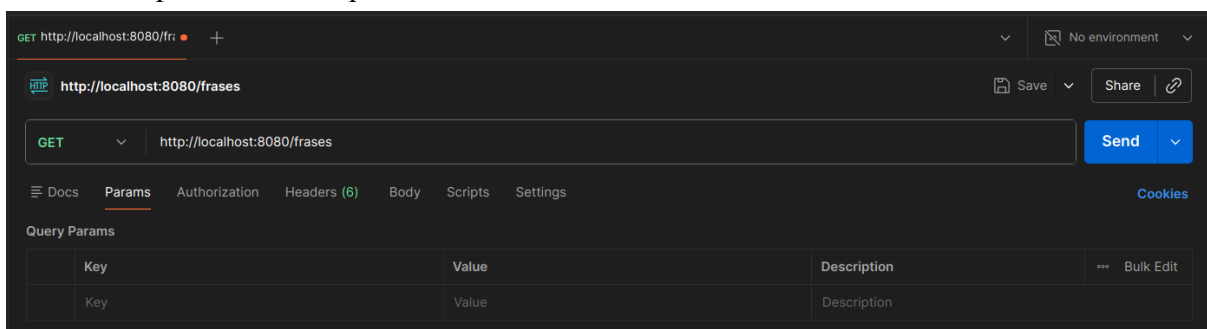
Ahora, en el navegador podemos entrar a la URL <http://localhost:8080/frases>. Y si vamos al modo Inspeccionar, y entramos a Network o Red, podemos ver la petición que se hizo al entrar a la URL y revisar que devuelve en formato JSON las mismas frases que cuando montamos la base de datos.

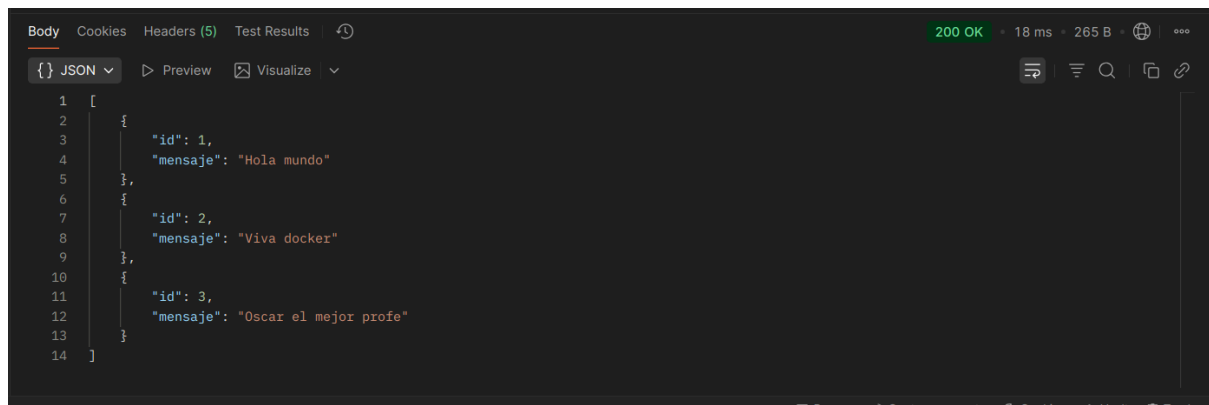


Estado	Método	Dominio	Archivo	Iniciador	Tipo	Transferido	Tamaño
200	GET	localhost:8080	frases	document	vnd.mozi...	263 B	108 B
	GET	localhost:8080	favicon.ico	img	json	NS_BINDING_ABORTED	0 B



También se puede hacer la prueba desde POSTMAN:





```
1 [
2   {
3     "id": 1,
4     "mensaje": "Hola mundo"
5   },
6   {
7     "id": 2,
8     "mensaje": "Viva docker"
9   },
10  {
11    "id": 3,
12    "mensaje": "Oscar el mejor profe"
13  }
14 ]
```

Errores comunes

Durante el desarrollo pueden presentarse algunos problemas frecuentes:

- **Docker no inicia correctamente:** puede ocurrir después de una instalación o reinstalación incompleta. Se recomienda verificar el servicio con `sudo systemctl status docker`.
- **Error de permisos al ejecutar Docker:** suele aparecer al usar comandos sin privilegios suficientes; en ese caso puede ejecutarse temporalmente con `sudo`.
- **La tabla no existe en PostgreSQL:** sucede cuando la base fue creada pero aún no se ha ejecutado el script SQL de inicialización.
- **Spring Boot no conecta a la base de datos:** generalmente se debe a errores en la configuración de `application.properties`, como puerto, usuario o contraseña incorrectos.
- **El backend no inicia desde terminal:** puede ocurrir si existen clases principales duplicadas o configuraciones anteriores del proyecto. En ese caso conviene revisar el paquete principal generado por Spring Initializr.
- **Puertos ocupados:** si los puertos 5432 o 8080 están siendo usados por otro proceso, la base de datos o el backend no podrán iniciar correctamente.

Conclusión

En este tutorial se construyó una aplicación backend básica utilizando Spring Boot y PostgreSQL, separando la base de datos en un contenedor Docker y ejecutando el backend localmente. Se logró crear la base de datos, inicializarla mediante scripts SQL y conectarla con una API REST desarrollada en Java mediante JPA/Hibernate.

Este ejercicio permite comprender el flujo básico de integración entre base de datos y backend, sirviendo como base para proyectos más complejos. A partir de este punto, el sistema puede ampliarse implementando operaciones CRUD completas, validaciones y la dockerización del backend para ejecutar toda la aplicación en contenedores., así como la implementación del frontend.